Machine Learning Fundamentals

POLYNOMIAL REGRESSION

Andy Louwyck
andy.louwyck@vives.be
Academic year 2024 - 2025

hogeschool vives

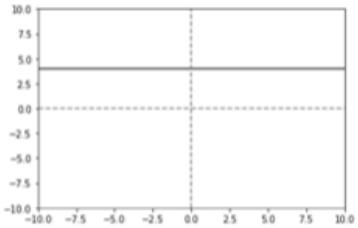
Polynomials

What if we need a curve instead of a straight line?

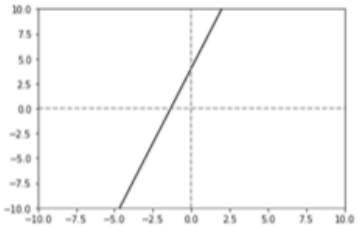
Polynomials

- $y = 4$
- $y = 3x + 2$ = linear equations
- $y = x^2 - 2x + 5$
- $y = 2x^3 + 8x^2 – 40$ = non-linear equations

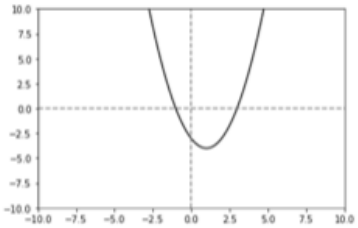
- **degree** = highest power
- max # of oscillations = degree - 1



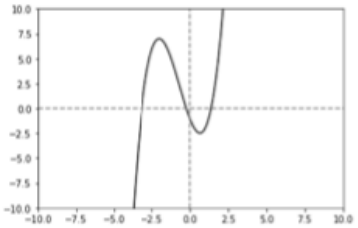
degree 0
$y = 4$



degree 1
$y = 3x + 4$



degree 2
$y = x^2 - 2x - 3$



degree 3
$y = x^3 + 2x^2 -4x - 1$

Useful?

- Example: 2 movies
  - One flew over the cuckoo's nest
  - 2 Fast 2 Furious
- Datum
  - x1: how long someone watches movie 1
  - x2: how long someone watches movie 2
- New movie:
  - 3 friends
- Wanted:
  - y: how long does someone watch movie 3?

Useful?

- Applying linear regression: $y = 0.5\ x1 + 0.3\ x2$

- The more users watch movie 1, the more they watch movie 3.
- Same goes for movie 2.
- But we see in the data that the more users watch movie 1 and movie 2, the less they watch movie 3.

- We can only express this with an additional term x1x2, for example:

  $y = 0.5\ x1 + 0.3\ x2 - \boxed{0.1\ x1x2}$ = non-linear term

Plotting polynomials: own function

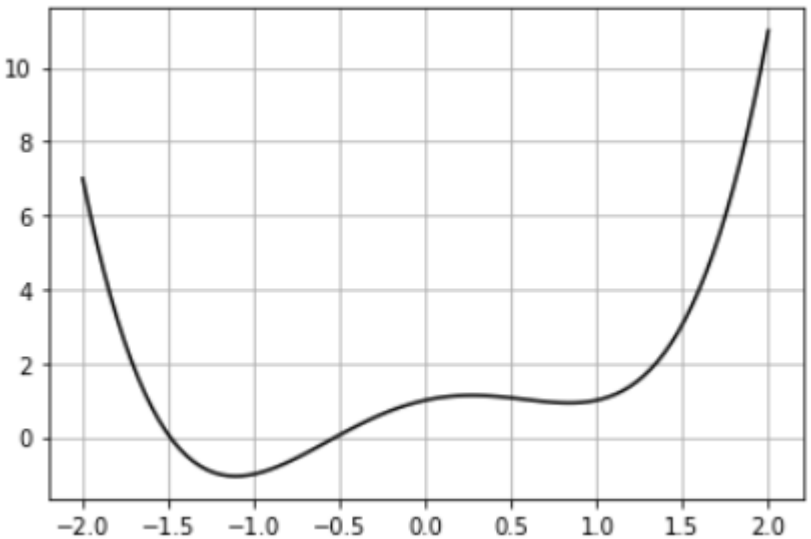
```
import numpy as np
import matplotlib.pyplot as plt

def polynomial(coefs, x):
    return sum([coefs[i]*x**i for i in range(len(coefs))])
```

$$y = x^4 - 2x^2 + x + 1$$

```
coefs = [1, 1, -2, 0, 1]
x = np.linspace(-2, 2, 100)
plt.plot(x, polynomial(coefs, x), 'k-')
plt.grid();
```

Coefficients from smallest to largest!



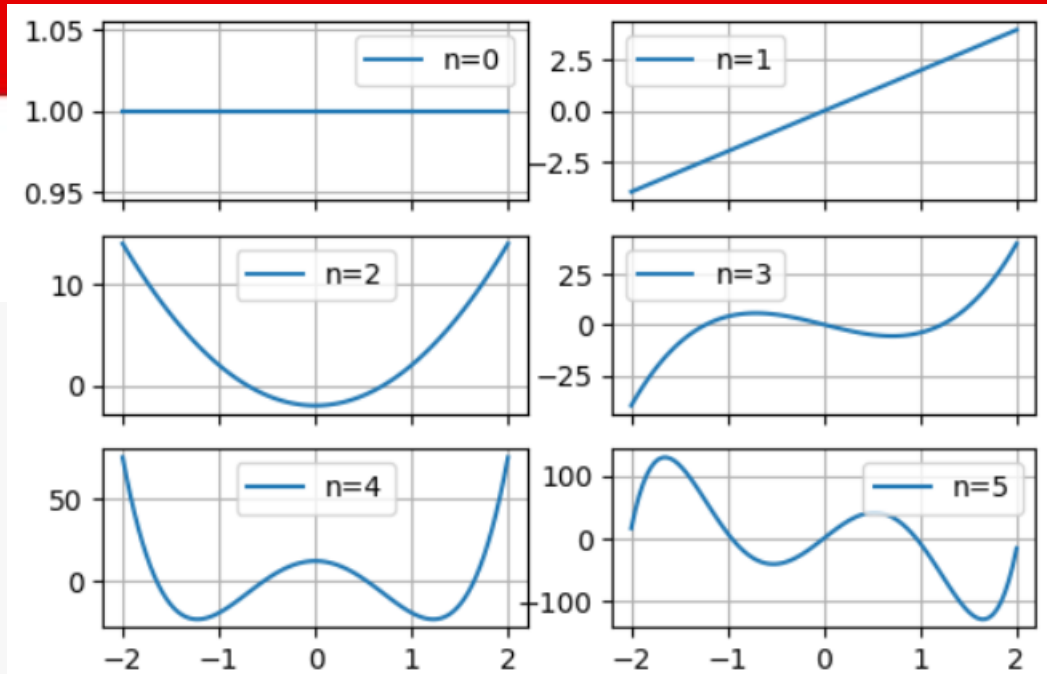
Plotting polynomials: NumPy function polyval

Coefficients from largest to smallest!

```
p = [[1],
     [2, 0],
     [4, 0, -2],
     [8, 0, -12, 0],
     [16, 0, -48, 0, 12],
     [32, 0, -160, 0, 120, 0]]
```

```
x = np.linspace(-2, 2, 100)

fig, ax = plt.subplots(nrows=3, ncols=2, sharex=True)
ax = [ax for row in ax for ax in row]
for i in range(len(p)):
    ax[i].plot(x, np.polyval(p[i], x));
    ax[i].grid();
    ax[i].legend([f'n={i}']);
```



$$p(x) = 1$$

$$p(x) = 2x$$

$$p(x) = 4x^2 - 2$$

$$p(x) = 8x^3 - 12x$$

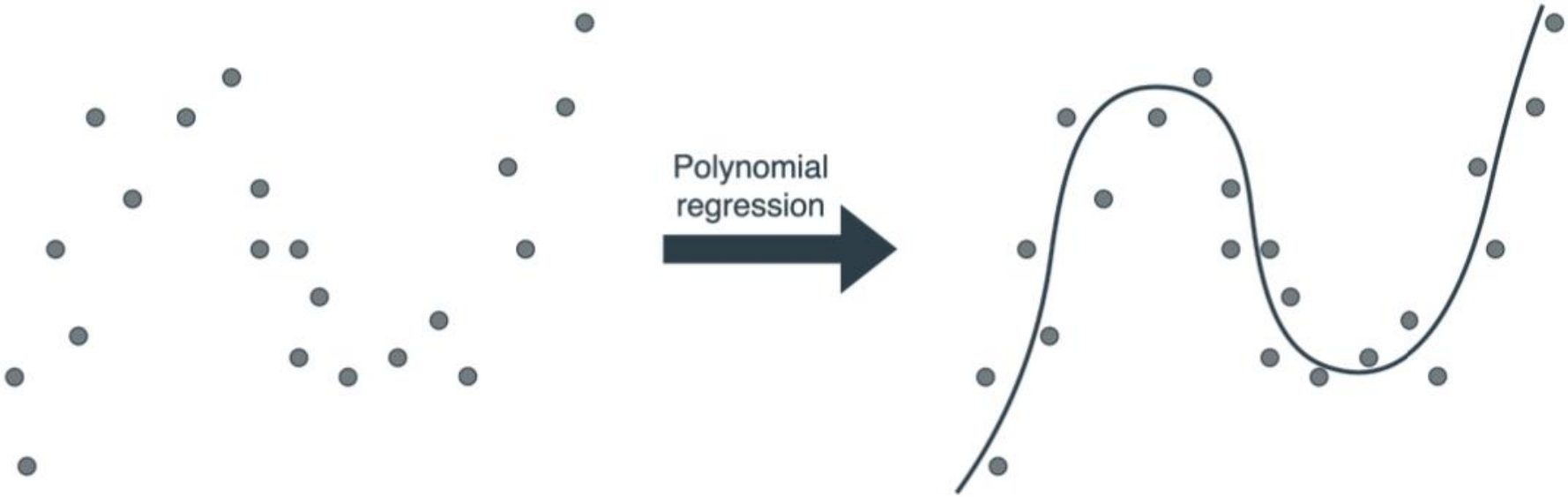
$$p(x) = 16x^4 - 48x^2 + 12$$

$$p(x) = 32x^5 - 160x^3 + 120x$$

Polynomial regression

What if the data points don't form a line?

Polynomial regression



Polynomial regression with NumPy: Function polyfit

```
# generating the dataset
x = np.linspace(-2, 2, 20);
y = np.polyval([32, 0, -160, 0, 120, 0], x)
np.random.seed(0)  # specify random seed so we always get the same result
y += np.random.randn(len(y))  # error with mean 0 and standard deviation 1
```

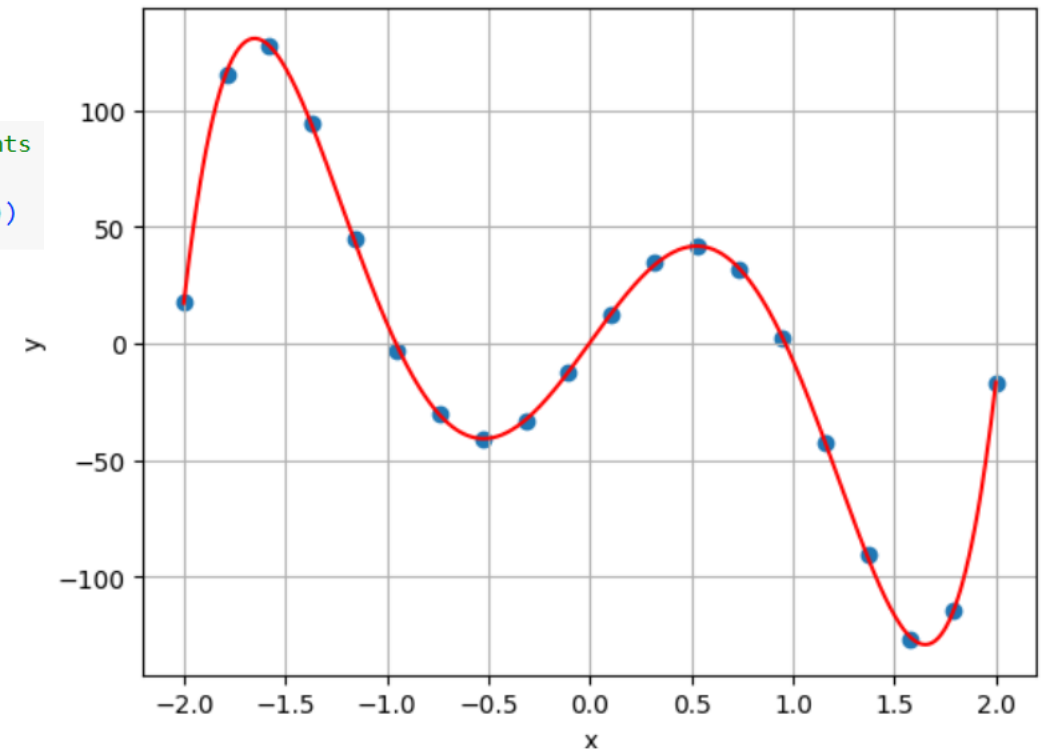
$$p(x) = 32x^5 - 160x^3 + 120x$$

```
# fitting the data
a = np.polyfit(x, y, deg=5)
```

```
yp = np.polyval(a, x)  # predictions of the data points
print("MAE:", np.round(np.mean(np.abs(y - yp)), 2))
print("MSE:", np.round(np.mean(np.square(y - yp)), 2))
```

```
MAE: 0.58
MSE: 0.49
```

```
xcurve = np.linspace(-2, 2, 500)
ycurve = np.polyval(a, xcurve)
plt.scatter(x, y);
plt.plot(xcurve, ycurve, 'r-');
plt.grid()
plt.xlabel('x');
plt.ylabel('y');
```



Polynomial regression in Scikit-Learn: PolynomialFeatures

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error

pr = make_pipeline(PolynomialFeatures(degree=5, include_bias=False),
                   LinearRegression())
pr.fit(X=x[:, np.newaxis], y=y)  # X must be 2D!
```

```
yp = pr.predict(x[:, np.newaxis])
print("MAE = {:.2f}".format(mean_absolute_error(y_pred=yp, y_true=y)))
print("MSE = {:.2f}".format(mean_squared_error(y_pred=yp, y_true=y)))
```

```
MAE = 0.58
MSE = 0.49
```

hogeschool vives

PolynomialFeatures explained!

```
degree = 5
x = x.reshape(-1, 1)  # reshape x to table with 1 column
X = np.hstack([x**n for n in range(1, degree+1)])  # create matrix X = [x, x**2, x**3, x**4, x**5]
pd.DataFrame(data=X, columns=['x', 'x^2', 'x^3', 'x^4', 'x^5']).head()  # convert to dataframe
```

	x	x^2	x^3	x^4	x^5
0	-2.000000	4.000000	-8.000000	16.000000	-32.000000
1	-1.789474	3.202216	-5.730281	10.254188	-18.349599
2	-1.578947	2.493075	-3.936434	6.215422	-9.813824
3	-1.368421	1.872576	-2.562473	3.506542	-4.798425
4	-1.157895	1.340720	-1.552413	1.797531	-2.081351

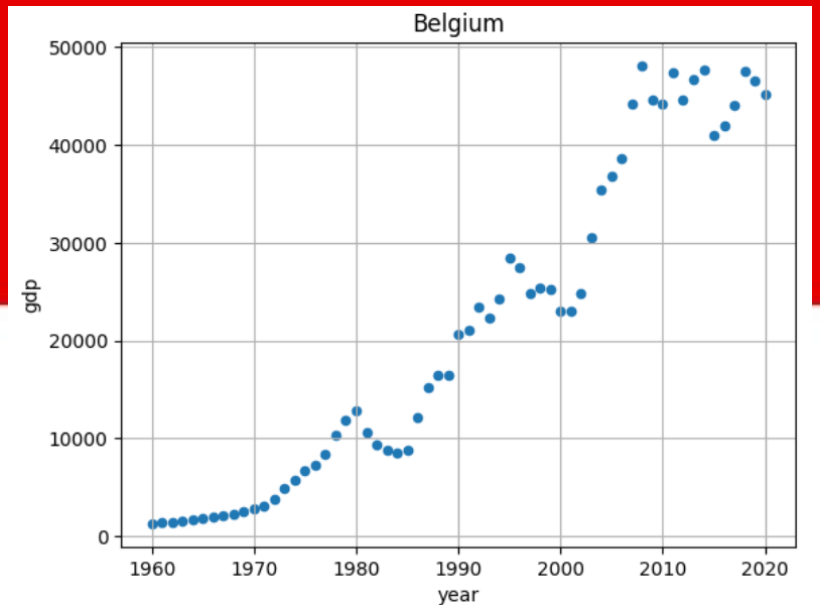
```
model = LinearRegression()
model.fit(X, y)
```

```
yp = model.predict(X)
print("MAE = {:.2f}".format(mean_absolute_error(y_pred=yp, y_true=y)))
print("MSE = {:.2f}".format(mean_squared_error(y_pred=yp, y_true=y)))
```

```
MAE = 0.58
MSE = 0.49
```

hogeschool vives

Polynomial Regression: Example with Real Data

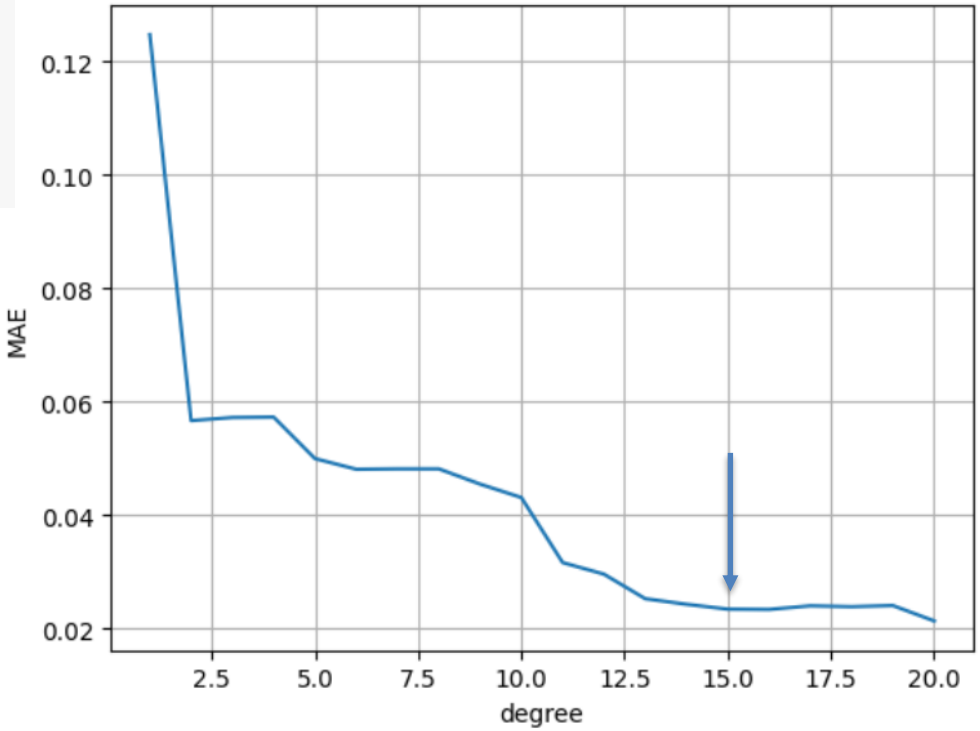


- GDP of Belgium from 1960 - 2020
- What polynomial degree?

```
x = belgium.year.values.squeeze() - 1960    # subtract 1960 from x
z = np.log10(belgium.gdp.values.squeeze())  # take logarithm of y

degrees = range(1, 21) # vary degrees from 1 to 20

MAE = []
for deg in degrees:
    a = np.polyfit(x, z, deg)
    zp = np.polyval(a, x)
    MAE.append(np.mean(np.abs(z - zp)))
```



- We choose degree 15

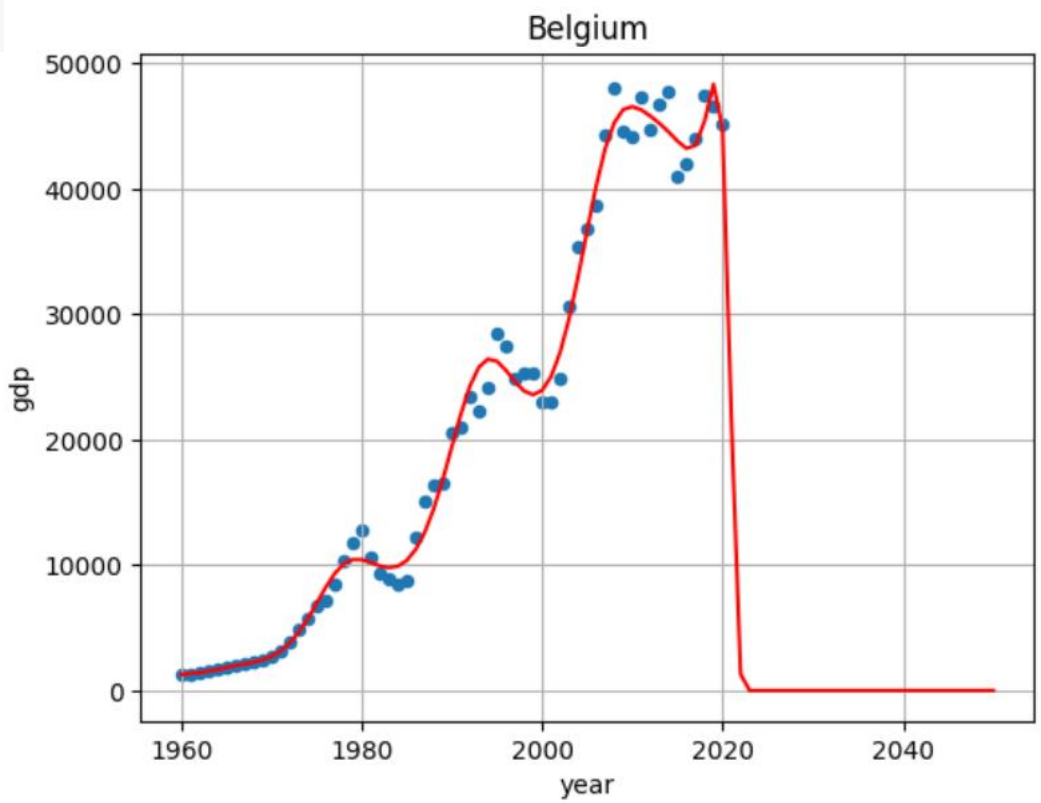
```
a = np.polyfit(x, z, deg=15)
zp = np.polyval(a, x)
yp = 10 ** zp  # don't forget to convert
```

Polynomial Regression: Example with Real Data

- Making predictions:

```
years = np.arange(1960, 2051)
gdp = 10 ** np.polyval(a, years - 1960) # don't forget to subtract 1960 from the years!

belgium.plot.scatter(x="year", y="gdp", title="Belgium");
plt.plot(years, gdp, 'r-');
plt.grid();
```



- It doesn't work!
  - Time series = ordered!

    don’t extrapolate!
  - Overfitting??

Underfitting and overfitting

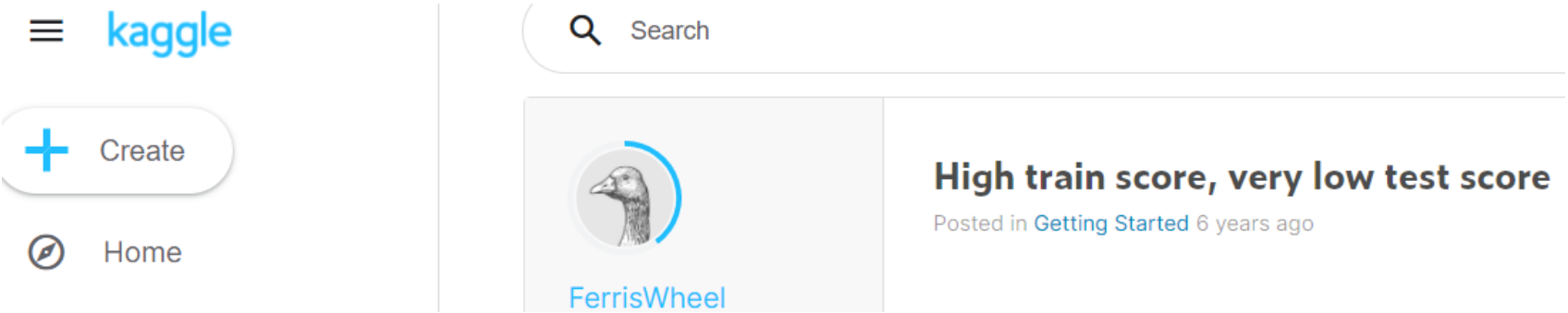
Or how your model should not be too generic,

But not too specific either…

Common scenario…

- You train an ML model
- You get high accuracy
- You integrate the model into software
- The model suddenly makes very bad predictions…

→ Underfitting or overfitting!!



What is underfitting and overfitting?

Compare to studying for an exam:

- You study too little: you have too little knowledge to solve the new exercises on the exam

  = UNDERFITTING

- You memorize everything: You do not understand the material and cannot apply it to solve the exam exercises

  = OVERFITTING

What is underfitting and overfitting?

- UNDERFITTING: your model is too simple, too generic
- OVERFITTING: your model is too complex, too specific

Underfitting

Overfitting

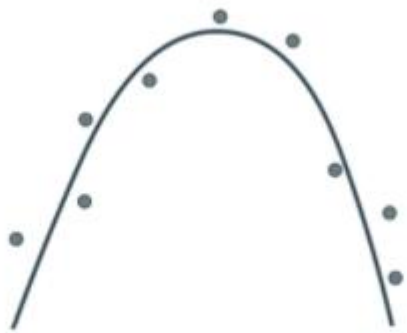
Underfitting and overfitting in regression

UNDERFITTING

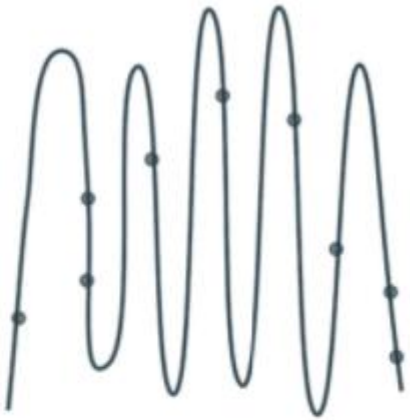
OK

OVERFITTING

Model 1
Polynomial of degree 1
(a line)

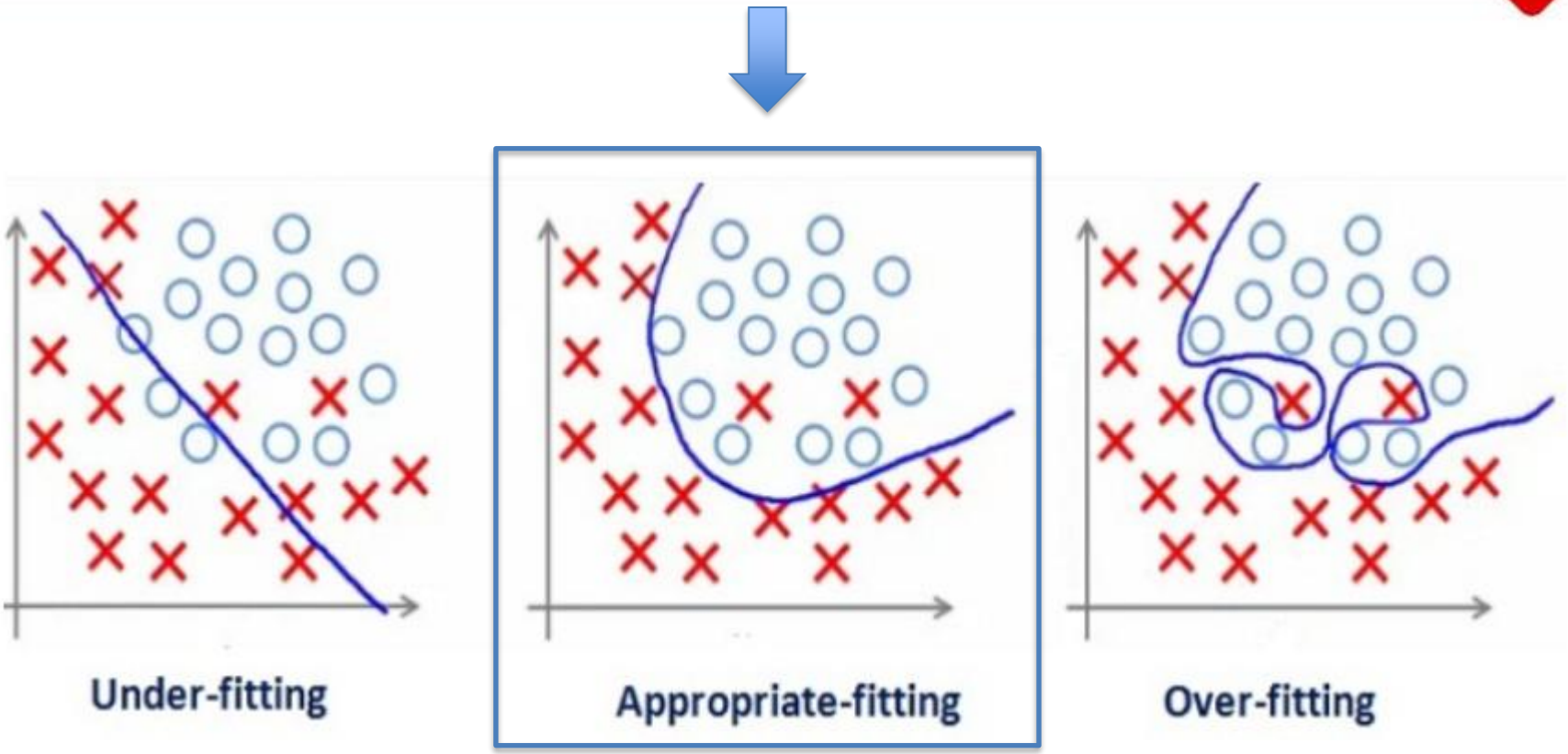


Model 2
Polynomial of degree 2
(a parabola)

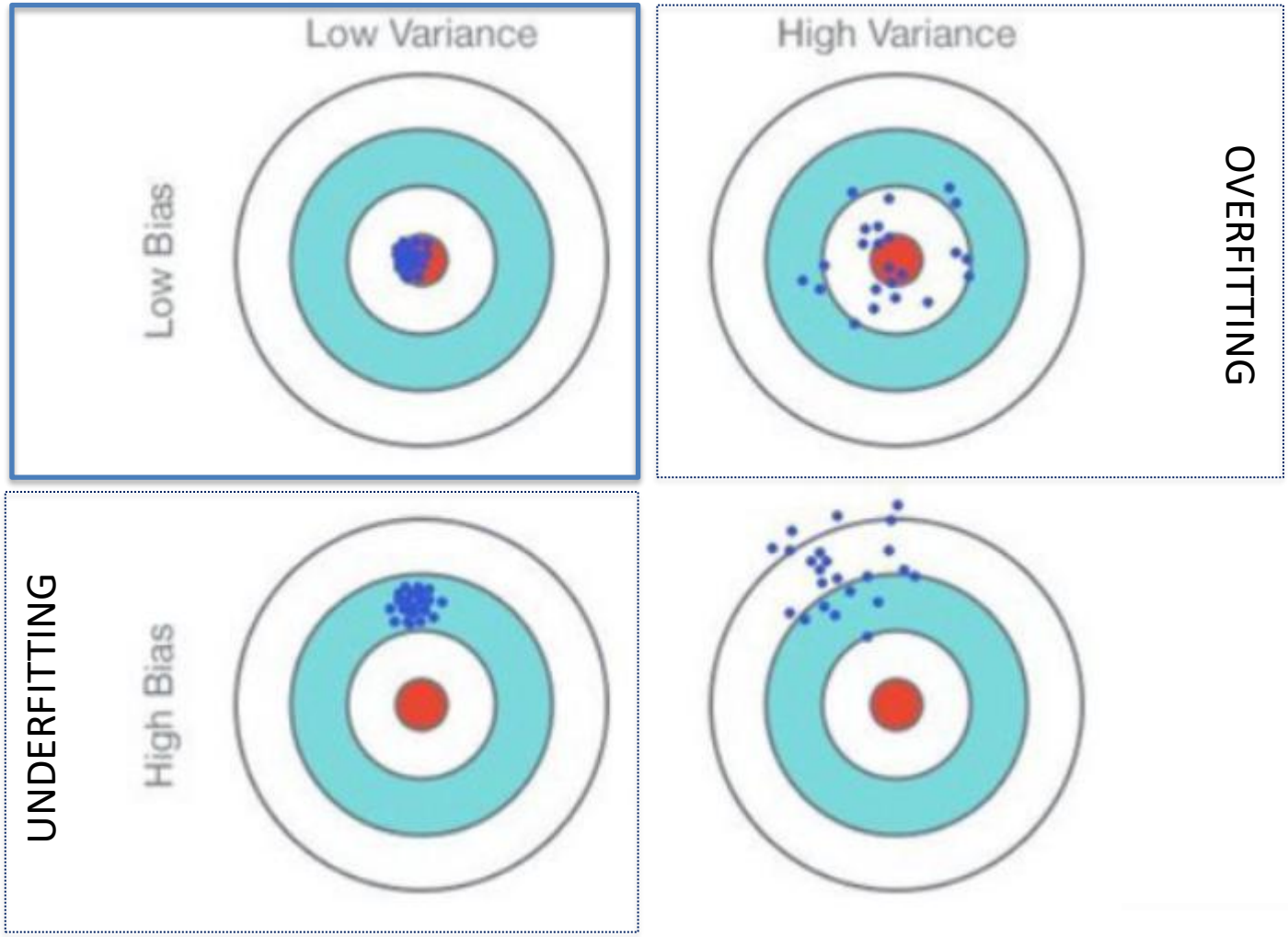


Model 3
Polynomial of degree 10

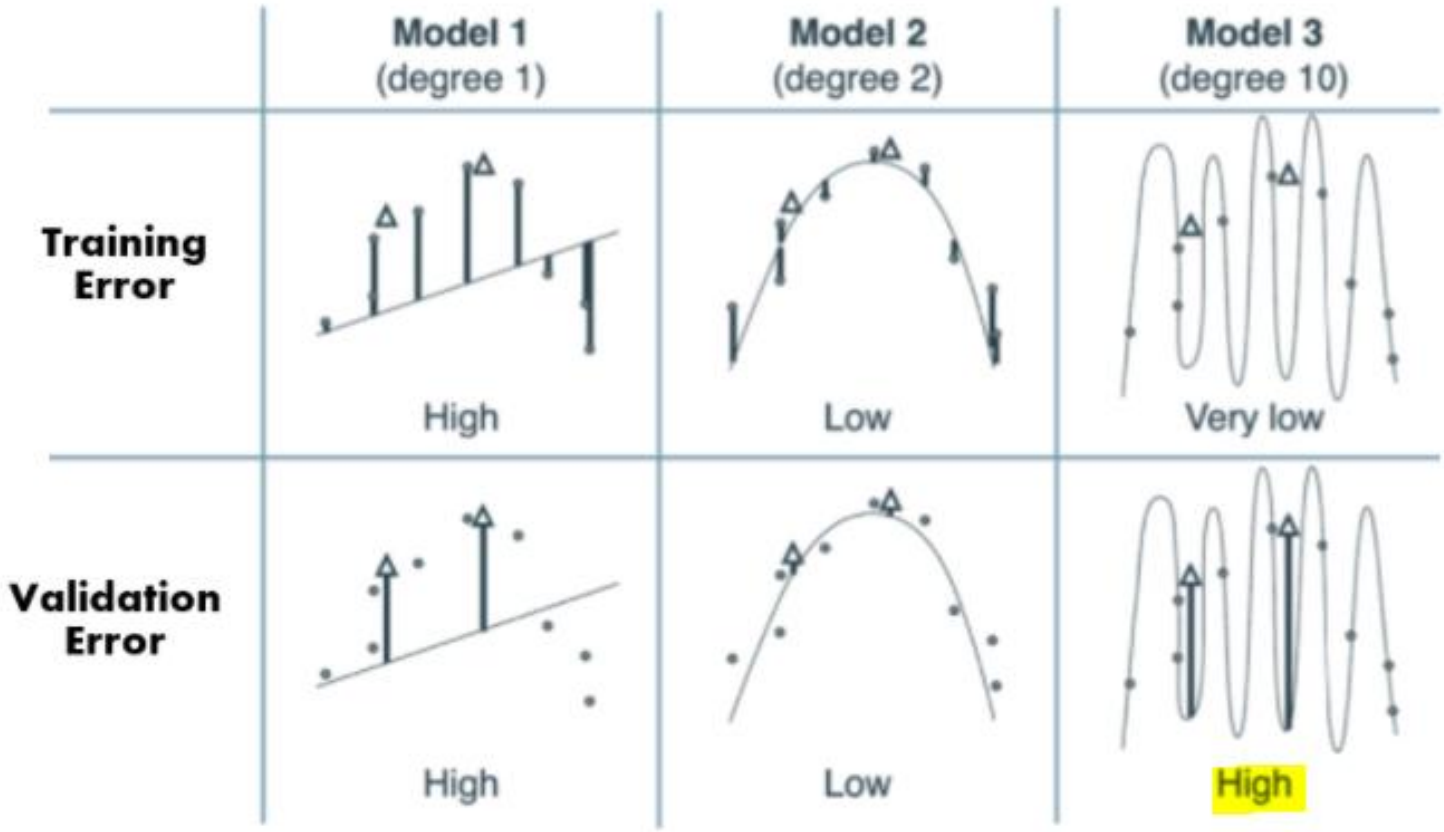
Underfitting and overfitting in classification



Bias – Variance Tradeoff



How to choose the right model? Validate!



How to prevent overfitting?

Splitting the dataset

The model complexity graph

Regularization

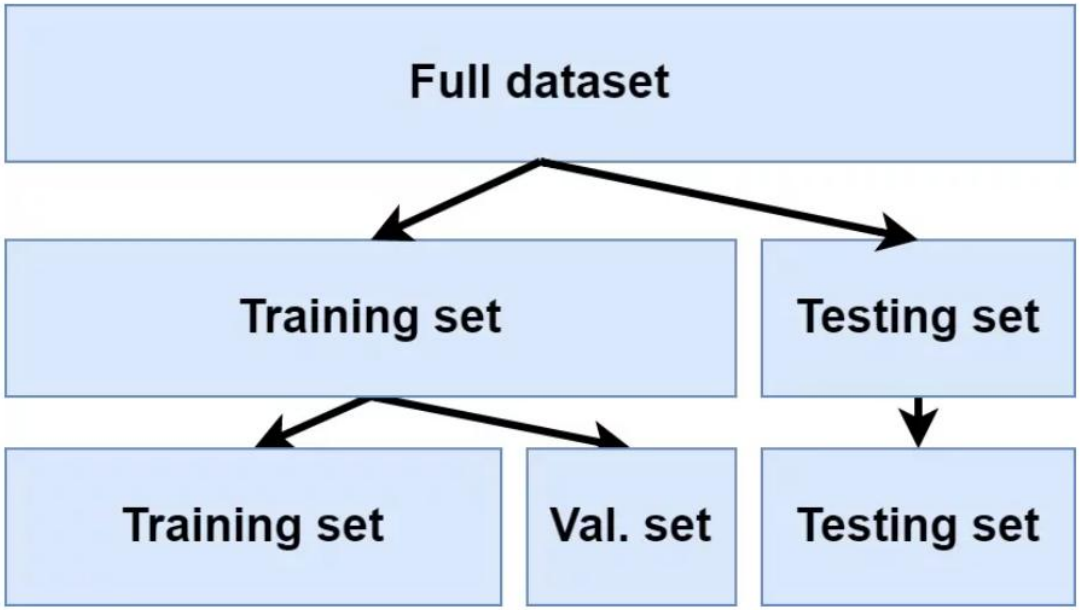
Splitting the dataset: Simple hold-out validation

- **Training set:** used for model training = optimize parameters
- **Validation set:**
  - Checking for overfitting
  - Fine-tuning hyperparameters (e.g. learning rate)
  - Choose the most suitable model

- **Test set:** Final evaluation

(= no information leaks)

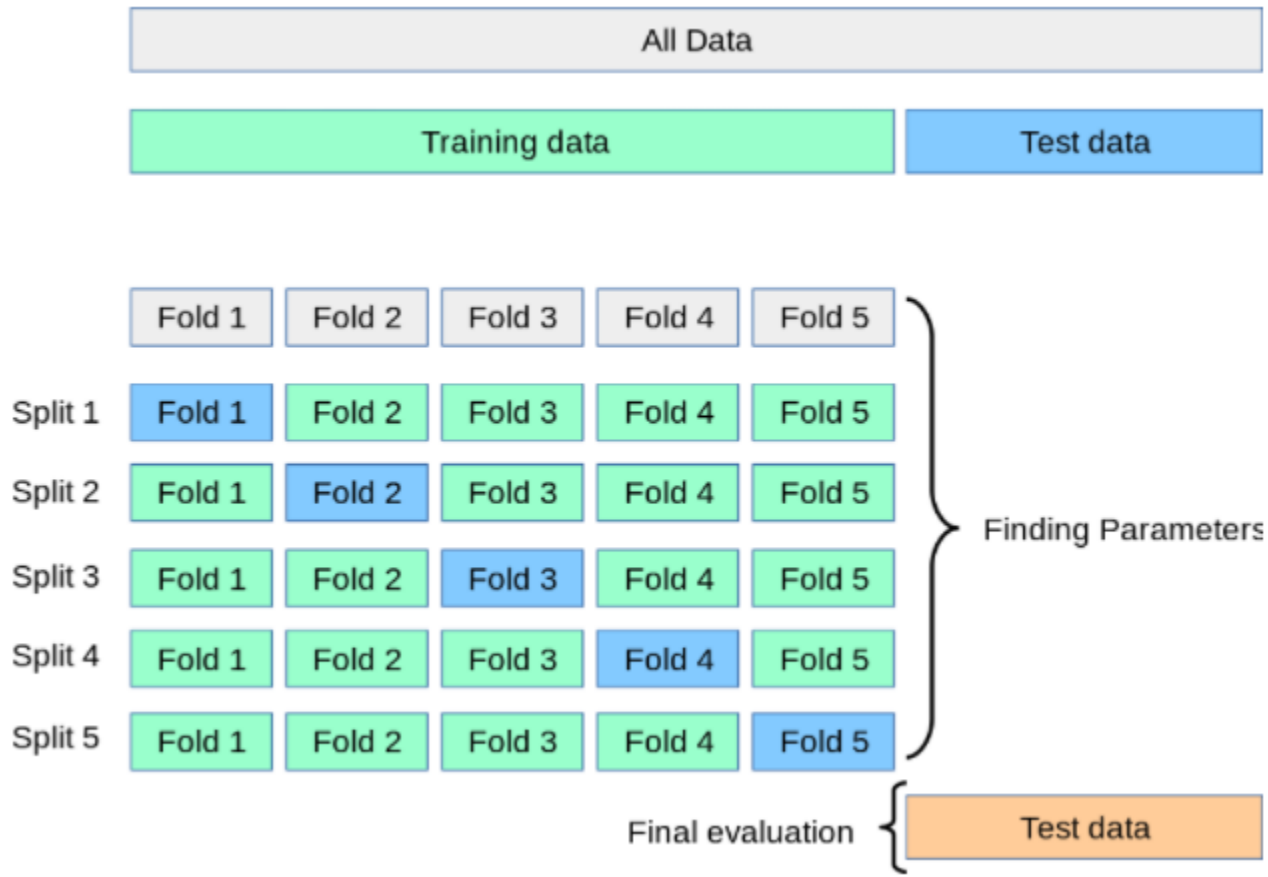
Simple hold-out validation



Splitting the dataset: K-fold cross-validation

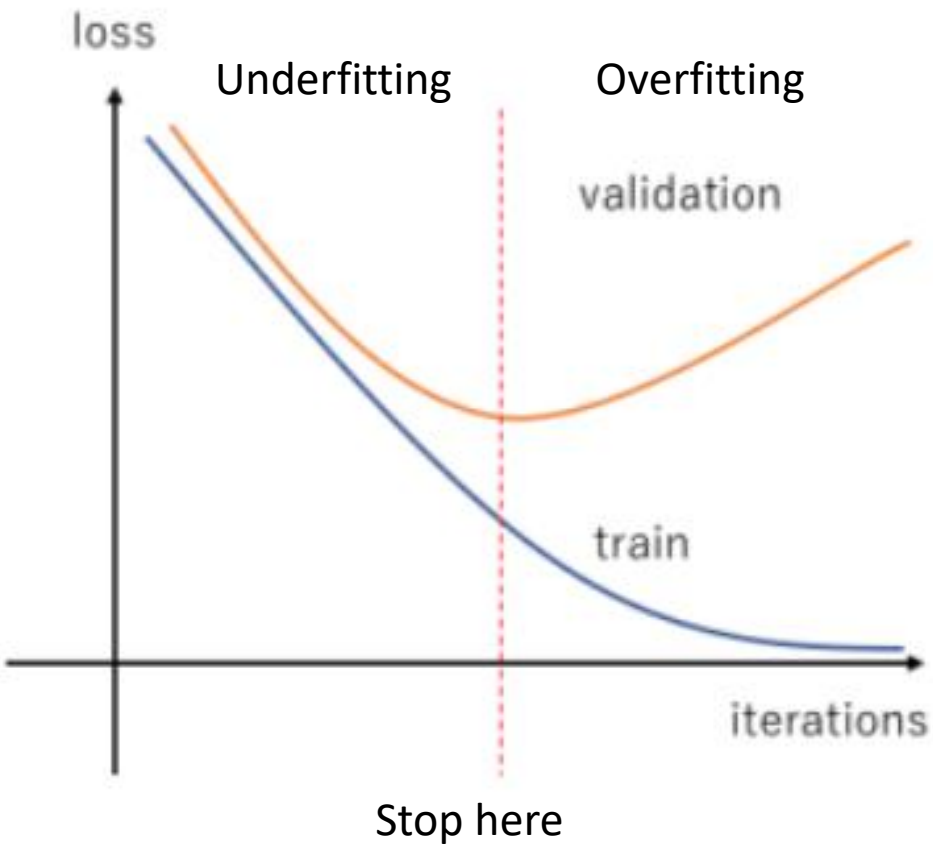
- Split training set into k “folds”
- k x train and validate + take average of k scores
- Useful for

  small data sets

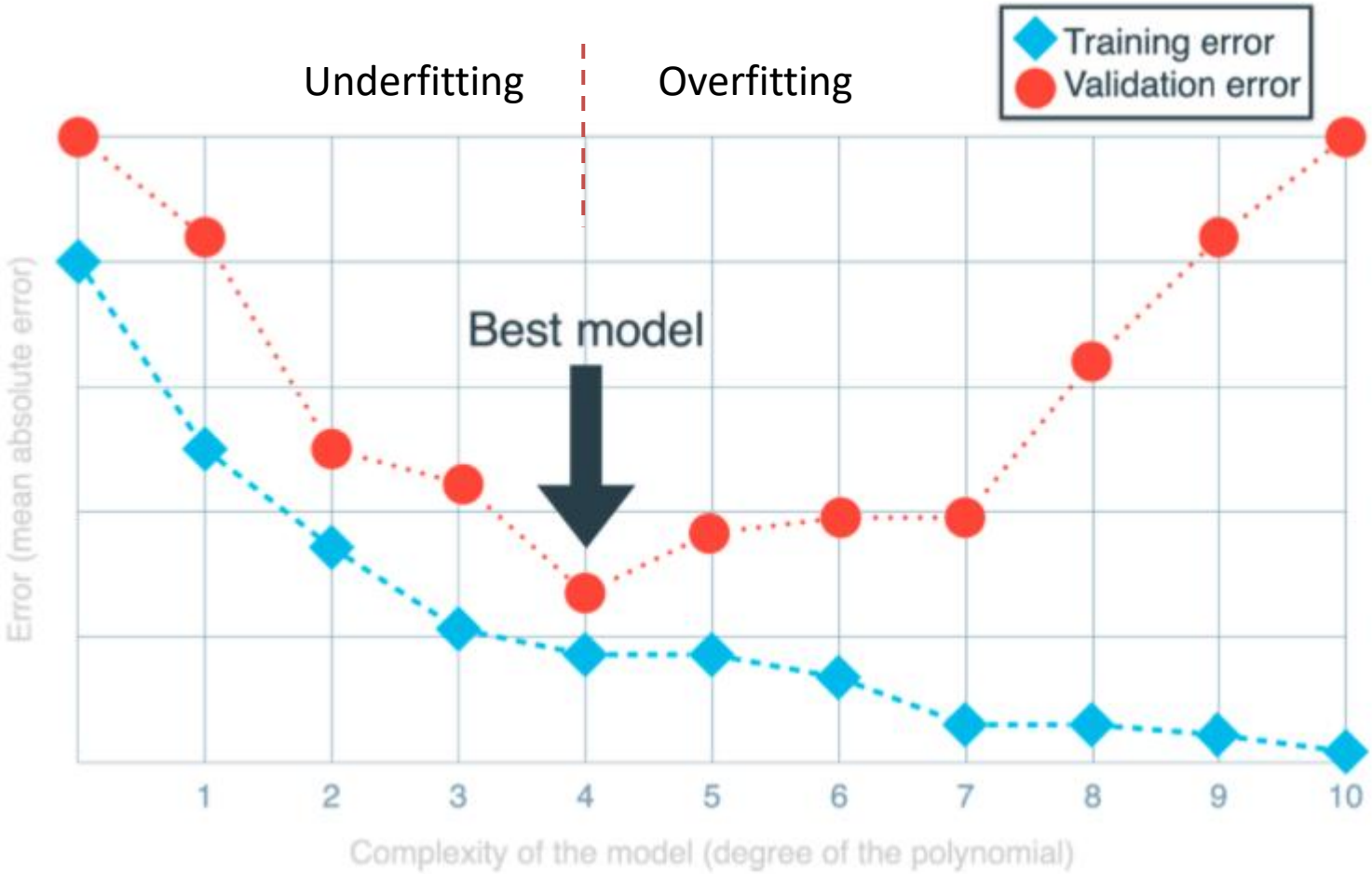


Validation set:
Checking for overfitting

Stop training when validation loss no longer improves…



In polynomial regression: The model complexity graph



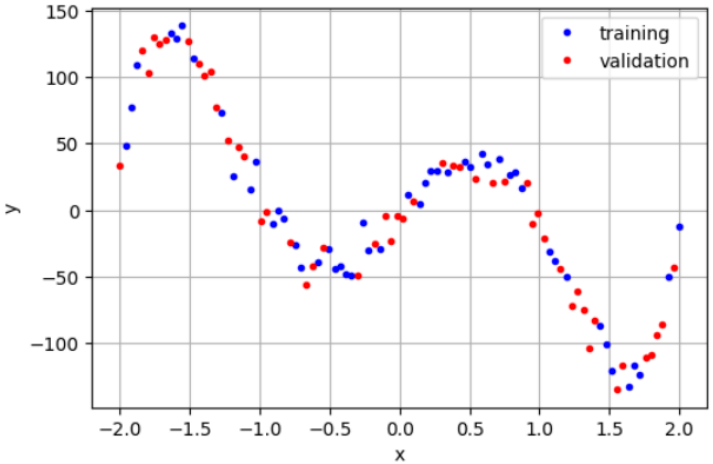
Model complexity graph: example

Generate dataset

```
x = np.linspace(-2, 2, 100);
y = np.polyval([32, 0, -160, 0, 120, 0], x)

np.random.seed(0)  # specify random seed so we always get the same result
y += np.random.randn(len(y)) * 10  # error with mean 0 and standard deviation 10
```

$$p(x) = 32x^5 - 160x^3 + 120x$$



Split dataset

```
from sklearn.model_selection import train_test_split
```

or

```
idx = np.arange(len(x))  # the indices of the data points
np.random.shuffle(idx)   # shuffle the indices
n = len(idx) // 2        # divide the dataset into two equal parts - be careful! use integer division!
ival = idx[:n]           # the indices of the data points in the validation set
itrain = idx[n:]         # the indices of the data points in the training set
```

Model complexity graph: example

Training and validating models

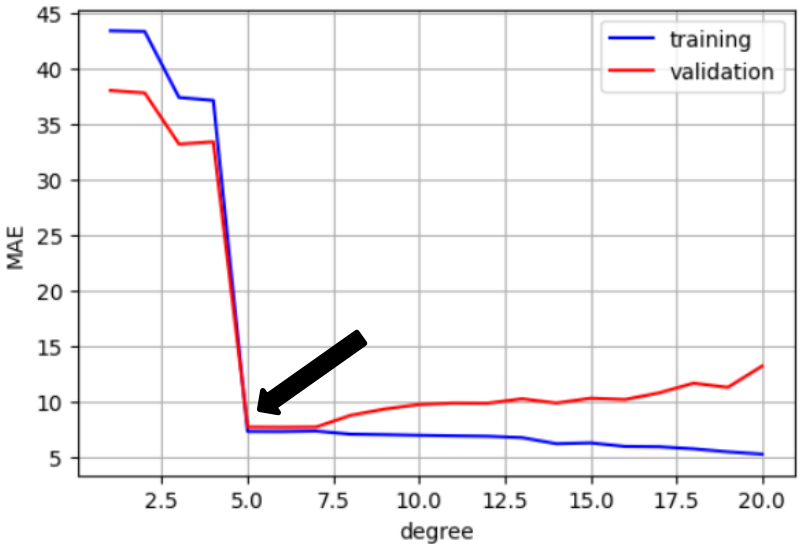
```
degrees = range(1, 21) # the degree of the polynomials

MAE_train = [] # initialize list for MAE of training data
MAE_val = [] # initialize list for MAE of validation data

for n in degrees:
    a = np.polyfit(x[itrain], y[itrain], deg=n) # fit only the training set!
    yp = np.polyval(a, x) # predictions of all data points
    MAE_train.append(np.mean(np.abs(yp[itrain] - y[itrain]))) # MAE of training data
    MAE_val.append(np.mean(np.abs(yp[ival] - y[ival]))) # MAE of validation data
```

Model complexity graph plotting

```
plt.plot(degrees, MAE_train, 'b-', degrees, MAE_val, 'r-')
plt.xlabel('degree');
plt.ylabel('MAE');
plt.grid();
plt.legend(['training', 'validation']);
```



Model complexity graph: example

Train final model with degree 5

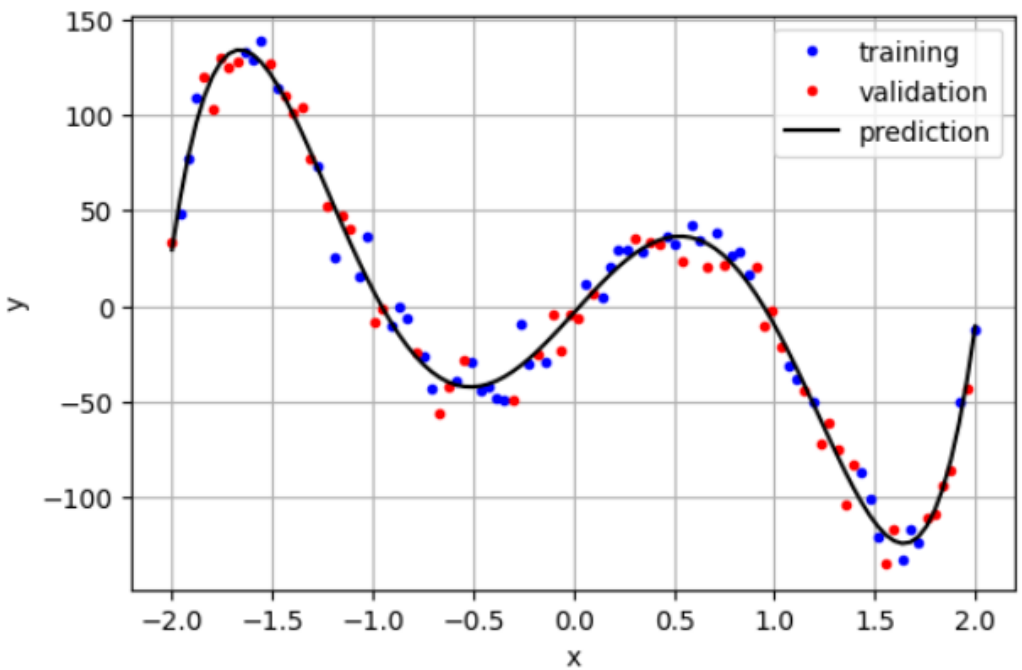
```
a = np.polyfit(x, y, deg=5)
a
```

```
array([ 3.12013880e+01,  1.00197087e-01, -1.56282920e+02,  2.88714278e+00,
        1.15854095e+02, -3.66288352e+00])
```

Evaluate final model

```
yp = np.polyval(a, x)
MAE = np.mean(np.abs(y - yp))
MAE
```

```
7.303895740027681
```



Regularization

- Many models are trained when composing the model complexity graph
- Regularization: improve model performance while training + keep the model as simple as possible
- Increase performance = minimize error function
- Keep it simple = minimize complexity function
- New cost function:

```
new_cost_function = performance_cost_function + complexity_cost_function
```

Analogy: the roofer

Problem: Broken roof

Roofer 1
Solution: Bandage
(Underfitting)

Roofer 2
Solution: Shingles
(Correct)

Roofer 3
Solution: Titanium
(Overfitting)

oplossing	lek (ml)	kostprijs (€)	TOTAAL
1	1000	1	**1001**
2	1	100	**101**
3	0	100000	**100000**

hogeschool vives

Example

- Linear regression with 10 features (= 11D problem):
  - Each feature = time a viewer watches a movie
  - Target = how long does a viewer watch movie 11?

$$y = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6 + w_7x_7 + w_8x_8 + w_9x_9 + w_{10}x_{10}$$

with:

- y the predicted time the user spends on movie 11;
- x_i the time the user spent on movie i;
- w_i the weight in the model corresponding to feature x_i;
- w_0 the bias of intercept.

Example

- 2 models as an example:

Model 1

$$y = 2x_3 + 1.4x_7 - 0.5x_9 + 4$$

Model 2

$$y = 22x_1 - 103x_2 - 14x_3 + 109x_4 - 93x_5 + 203x_6 + 87x_7 - 55x_8 + 378x_9 - 25x_{10} + 8$$

- Model 2 is clearly the most complex:
  - many weights (= number of parameters)
  - large weights (= parameter values)
- We prefer model 1!
- But how to express complexity mathematically?

L1 and L2 norm

- L1 norm = sum of the absolute values of the coefficients
- L2 norm = sum of the squares of the coefficients

Model 1

- L_1-norm: $|2| + |1.4| + |-0.5| = 3.9$
- L_2-norm: $(2)^2 + (1.4)^2 + (-0.5)^2 = 6.21$

Model 2

- L_1-norm: $|22| + |-103| + |-14| + |109| + |-93| + |203| + |87| + |-55| + |378| + |-25| = 1089$
- L_2-norm: $(22)^2 + (-103)^2 + (-14)^2 + (109)^2 + (-93)^2 + (203)^2 + (87)^2 + (-55)^2 + (378)^2 + (-25)^2 = 227131$

- Model 2 has higher L1 and L2 norms

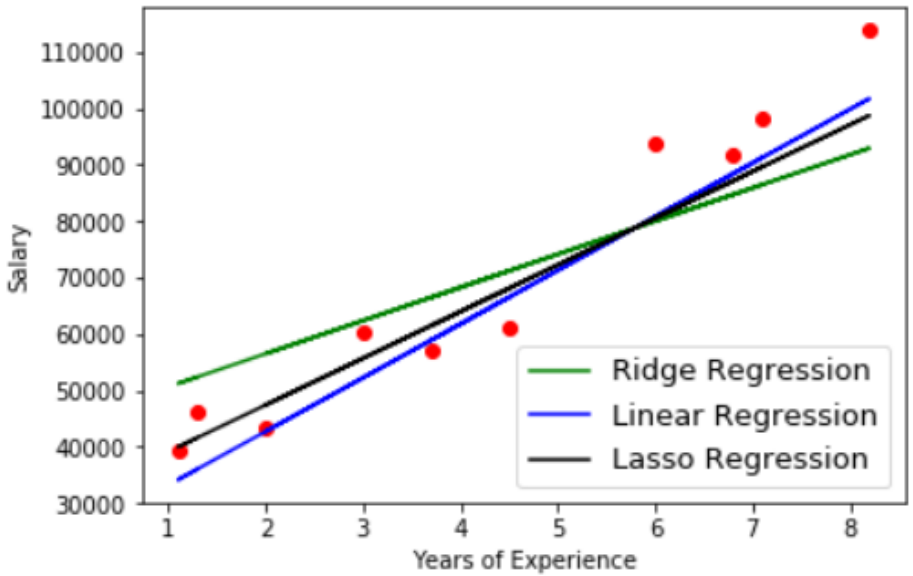
  → Model 2 is the most complex!
- Please note: DO NOT include intercept/bias!

Applying L1 and L2 norm in regression

- Lasso : cost = loss + λ * L1 norm
- Ridge : cost = loss + λ * L2 norm

= regularization

- L1 or L2 norm = regularization cost
- Loss = regression cost
- λ = regularization parameter
  - $\lambda = 0$: the regularization cost is not included;
  - $\lambda = 0.01$ or $\lambda = 0.1$: part of the regularization cost is included;
  - $\lambda = 10$ or $\lambda = 100$: a multiple of the regularization cost is included.



Effect of Lasso and Ridge Regularization

- Lasso regularization:
  - Coefficients can become zero
  - E.g., recommender systems

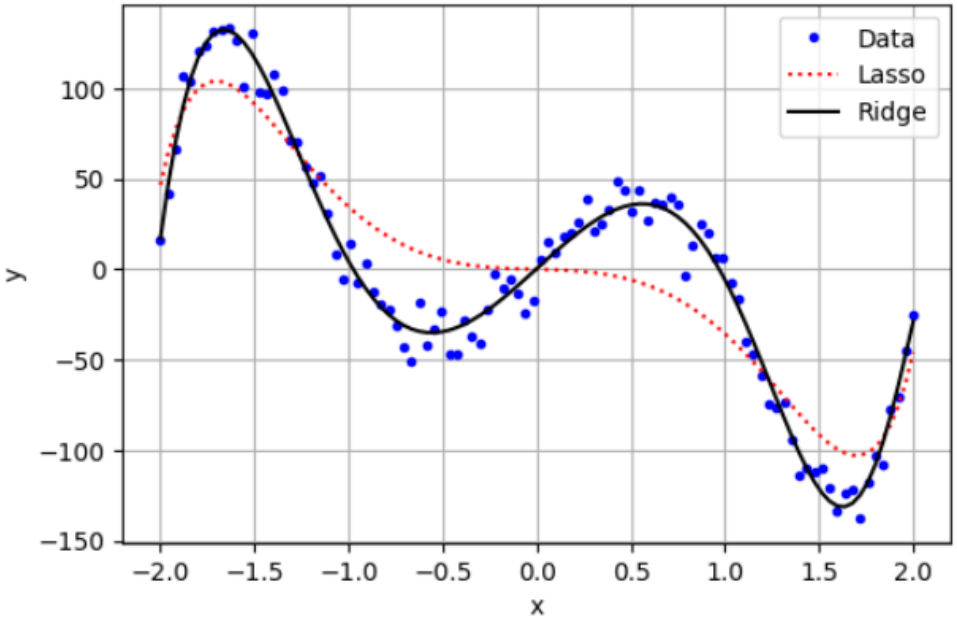
- Ridge regularization:
  - Coefficients can become small but not zero
  - E.g., image recognition

Lasso and Ridge Regularization with Scikit-Learn

```
from sklearn.linear_model import Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

degree = 10 # polynomial of degree 10
lasso = make_pipeline(PolynomialFeatures(degree, include_bias=False), # bias not needed!
                      Lasso(alpha= 0.1, tol=0.1))  # Lasso regression
lasso.fit(x[:, np.newaxis], y);

ridge = make_pipeline(PolynomialFeatures(degree, include_bias=False), # bias not needed!
                      Ridge(alpha=0.1, tol=0.1))  # Ridge regression
ridge.fit(x[:, np.newaxis], y);
```



Sources

- Serrano, L. G., 2020. Grokking Machine Learning, Manning Early Access Program, Manning Publications.
- Chollet, F. 2021. Deep Learning with Python (2nd edition), Manning Publications.
- Online Scikit-Learn documentation:

  https://scikit-learn.org/stable/
- Online NumPy documentation:

  https://numpy.org/doc/stable/

hogeschool vives